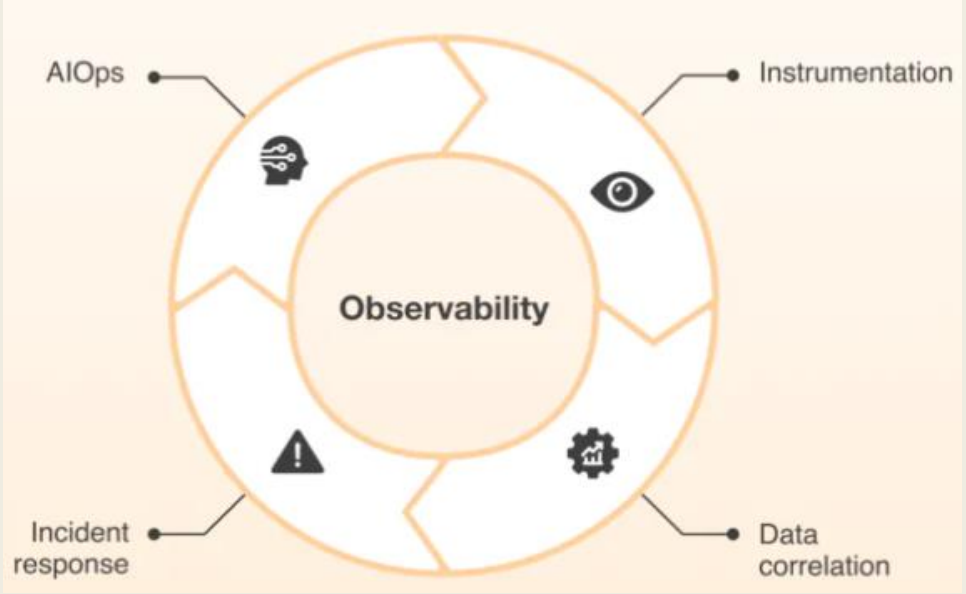
Agent Observability

- **What is observability?**
  - Observability is the ability to understand a system's internal behavior **just by looking at its outputs**.
  - To make a system observable:
    - **Instrument the code:** Add monitoring tools so the system can generate information such as Metrics, Logs, Traces, Events.
    - **Data correlation and context:** Combine data from various entities to:
      - Establish a correlation
      - Spot patterns
      - Identify unusual behavior
    - **Incident response:** When something breaks:
      - Alerts are generated
      - The right team is notified
      - They can act quickly to fix the issue
    - **AIOps:** Machine learning models automatically aggregate, correlate, and prioritize incident data.
      - Filter out unrelated data and create an alert signal or detect issues that can impact the system's performance and efficiency.



Agent Observability

- **When Do We Need Agent Observability**
  - Debug Agent Behavior
    - Understand **why** the agent made a decision
    - See which step failed
  - Track Tools Calls
    - Know which tools were called
    - Detect tool failures
  - Monitor Performance
    - Measure response time
    - Identify slow reasoning or slow tools
  - Detect Errors
    - Catch failed tool calls
    - Detect Retries and timeout
  - Improve Agent Quality
    - Identify hallucination or wrong decision

Agent Observability

- **What is OpenTelemetry?**
  - OpenTelemetry is an **open-source observability framework** that helps you generate, collect, and export **telemetry data** from applications.
  - Telemetry means:

Pillar	Description	Example
Traces	End-to-end path of a request	API -> DB -> Cache
Metrics	Numeric values over time	CPU, latency, memory
Logs	Text messages describing events	Error, warnings, audits

Agent Observability

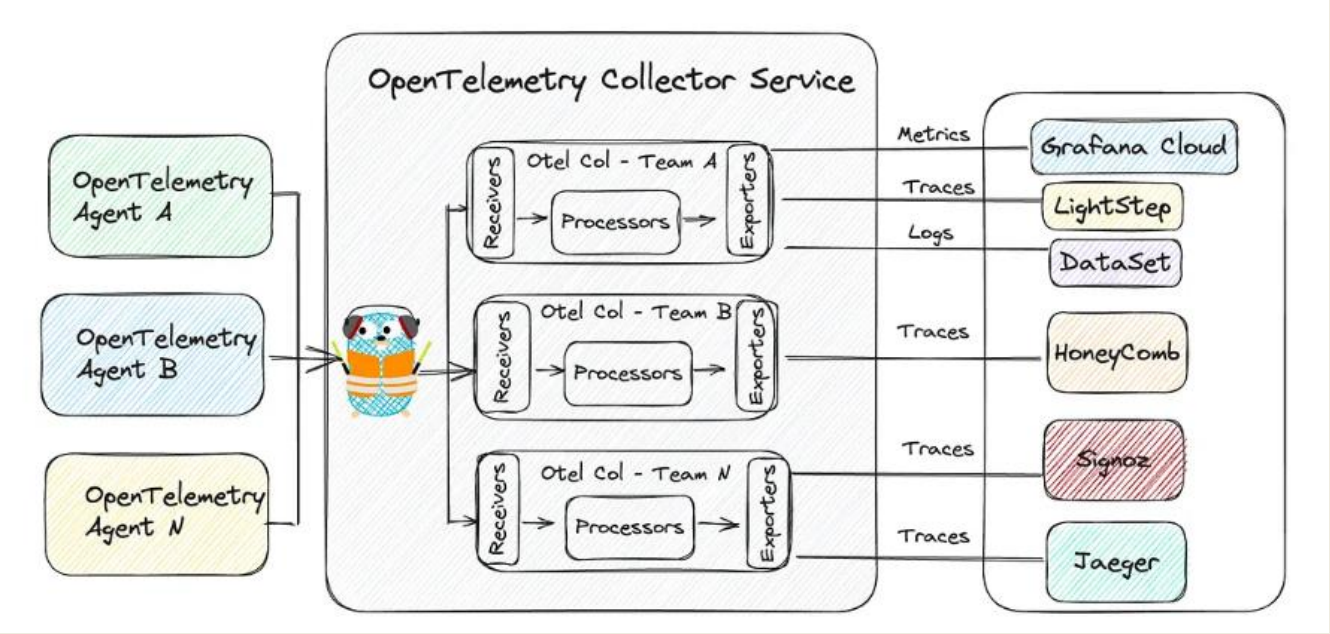
- **Why OpenTelemetry?**
  - **The Problem:**
    - Modern systems have large, complex architectures that include:
      - Multiple Microservices
      - Containerized Workloads
      - Cloud Infrastructure
      - Distributed components
    - It’s hard to understand **what’s happening** inside them.
  - **How Open Telemetry Solves This:**
    - ***Vendor-neutral*** **-** Works with any cloud, any language, any tool (No lock-in).
    - ***You own your data*** **-** All telemetry stays under your control.
    - ***One universal standard*** **-** Same API for **logs + metrics + traces** → less confusion.
    - ***Works everywhere*** **-** Supports most languages, frameworks, and runtimes.

Agent Observability

- **Core Components of OpenTelemetry**
  - **SDK**
    - Language-specific libraries
    - Eg: Python, Node.js
  - **Instrumentation Libraries**
    - Auto-instrumentation (HTTP, DB calls)
  - **Collectors**
    - Vendor-neutral pipeline
      - Received telemetry
      - Process (filter, sample, convert)
      - Sends it to backends
  - **Exporters**
    - Send data to
      - **Jeager -** open-source distributed tracing system used to visualize request flows and diagnose latency issues.
      - **Grafana Tempo -** OpenTelemetry-native distributed tracing backend designed for cheap, long-term trace storage.
      - **Prometheus -** open-source metrics collection and monitoring system that stores time-series data for dashboards and alerts.
      - **CloudWatch -** Amazon's cloud-native monitoring and logging service for metrics, traces, logs, and alerts across AWS applications.

Agent Observability

- **Core Components of OpenTelemetry**
  - **Exporters**
    - Send data to
      - **Datadog -** A commercial observability platform that provides unified metrics, logs, traces, and dashboards with AI-driven insights.
      - **Elasticsearch (Elastic Stack) -** A search and analytics engine often used for storing and querying logs, metrics, and operational data.
      - **New Relic -** A full-stack observability platform offering application performance monitoring (APM), metrics, logs, and traces.



Agent Observability

- **How OpenTelemetry facilitates instrumentation?**
  - **Step 1: Add OpenTelemetry packages to your code**
    - These packages give you:
      - The OTel API (to create spans, metrics)
      - The OTel SDK (the engine behind the scenes)
      - The OTLP exporter (to send data to the OTel Collector)
      - Auto-instrumentation tools

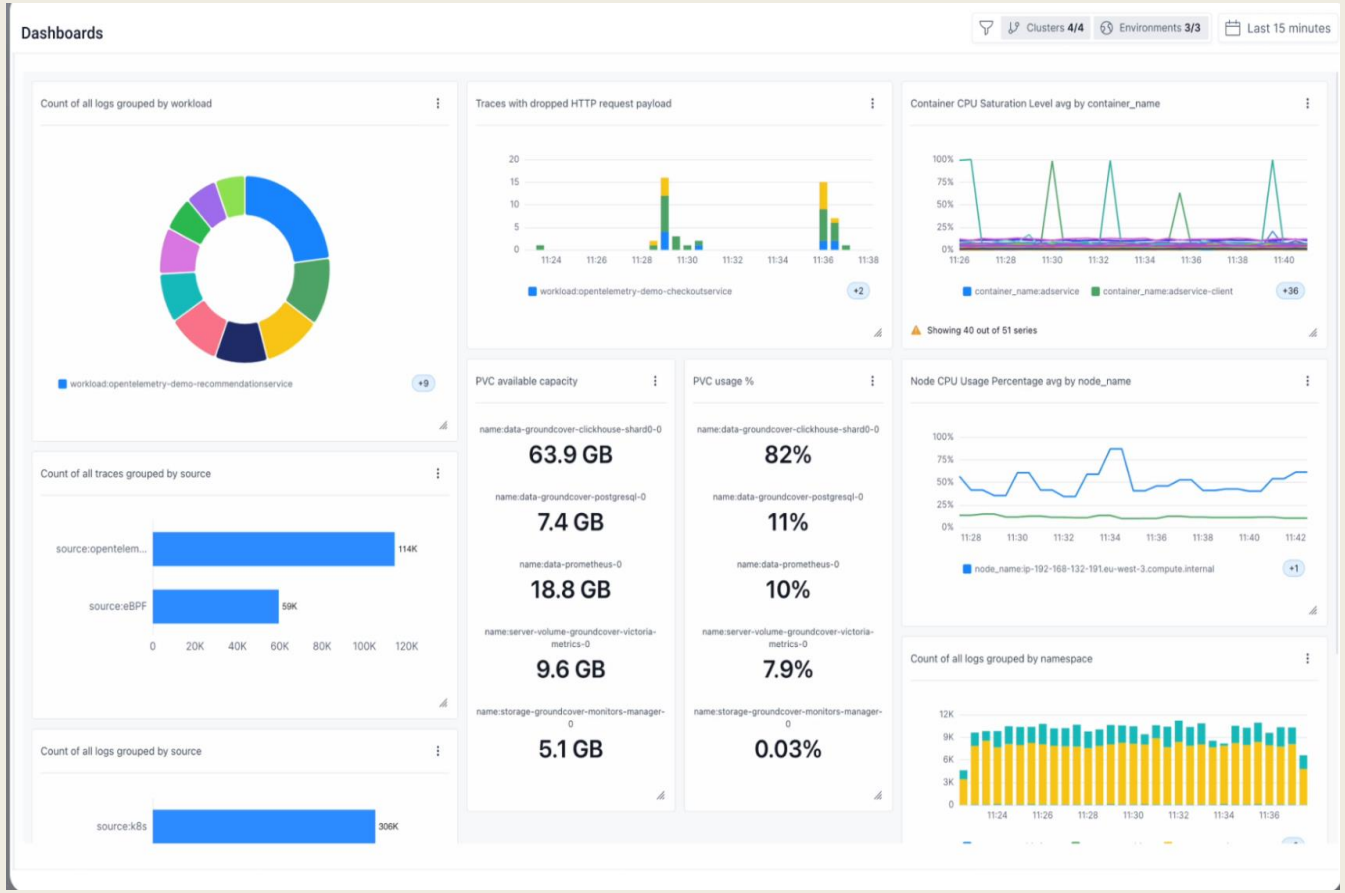
```
pip install opentelemetry-sdk
pip install opentelemetry-api
pip install opentelemetry-exporter-otlp
pip install opentelemetry-instrumentation
```

  - **Step 2: Instrument code**
    - You add instrumentation to:
      - a function
      - a calculation
      - an API call
      - a DB query

```
with tracer.start_as_current_span("calculate_score"):
    result = score_function()
```

Agent Observability

- **How OpenTelemetry facilitates instrumentation?**
  - **Step 3: Code automatically emits telemetry**
    - Once instrumented, your applications can emit:
      - **Traces -** which functions ran
      - **Metrics** - how long they took
      - **Logs** - important events
  - **Step 4: The telemetry is sent to OpenTelemetry Collector**
  - **Step 5: Collector forwards the data to tools**
    - The Collector can send your code's telemetry to :
      - Jeager
      - Cloudwatch
      - Datadog
      - Any backend



Agent Observability

- **Why Agents Need Observability**
  - AI Agents behave differently from traditional apps:
    - Understand what steps the agent took (trace flows)
    - Measure how well it performs (metrics)
    - Record what happened (logs & events)
    - Detect and alert on failures or anomalous behavior
    - Debug reasoning chains (LLM steps + tool calls)
    - Audit HITL interactions and approval
  - **Observability becomes critical** to debug + monitor agent behavior.
- **What to Observe in an AI Agent**

Category	What to track	Why
Traces	Tool calls, model steps, sub-agents	Understand reasoning and execution flow
Metrics	Latency, token usage, cost	Performance & Cost optimizations
Logs	Prompts, errors, retries	Debugging & audits
Events	HITL approvals, interruptions	Workflow reliability

Agent Observability

- Build Agents with Observability:
  - **Step 0 – Prerequisites Agent codebase (Python recommended for examples).**
    - OpenTelemetry packages installed.
    - A collector / backend ready (e.g., local OTLP Collector + Grafana + Prometheus + Jaeger).
  - **Step 1 – Design your telemetry model (what to capture)**
    - Decide what events, trace, spans, and metrics agent must emit.
    - **Traces**:
      - Full journey of one agent request
      - Example: User -> Agent -> LLM -> Tool -> Validation -> Response
      - Create a tracer:

```
from opentelemetry import trace
tracer = trace.get_tracer(__name__)
```

    - **Span**:
      - One step inside that request (plan, LLM call, tool call, validation)
      - Each span has Start time, End time, Status (success/failure), Metadata (agent info, LLM info, Tool info, Errors, HITL)

Span Type	Where it goes
Root span	Around the agent's main run() function
Child spans	Around each logical step inside the agent